ELSEVIER

European Journal of Operational Research 139 (2002) 578–584

EUROPEAN JOURNAL OF OPERATIONAL RESEARCH

www.elsevier.com/locate/dsw

Discrete Optimization

Solving some lexicographic multi-objective combinatorial problems

A. Volgenant *

Operations Research Group, Faculty of Economics and Econometrics, University of Amsterdam, Roetersstraat 11, 1018 WB Amsterdam, The Netherlands

Received 22 November 1999; accepted 6 June 2001

Abstract

The lexicographic multi-objective optimization problem (r-LMOP) with r (sum or bottleneck) objectives optimizes a first objective, then as far as a choice remains, a second one, then a third one and so on. A general solution scheme from literature is based on scaling; for the case that only sum criteria are involved the complexity is $O(r \log n)T(n,m)$ with $T(n,m)$ the complexity of optimization on combinatorial (sum) problem instances with m arcs and n nodes. For cases with at least one bottleneck criterion the complexity is at least $O(rn \log^2 n)T(n,m)$. We describe a solution scheme that is suited to solve the shortest path r-LMOP, the minimum spanning tree r-LMOP and the linear assignment r-LMOP; the complexity is in all cases $O(r)T(n,m)$. © 2002 Elsevier Science B.V. All rights reserved.

Keywords: Integer programming; Lexicographic multi-objective criterion; Linear assignment; Shortest path; Minimal spanning tree

1. Introduction

Instances of combinatorial optimization problems have often a set of optimal solutions that contains a lot of elements, especially problem instances with bottleneck criteria; it can be attractive to apply further criteria that express preferences among these solutions. For a number of combinatorial optimization problems we consider how to determine solutions that optimize a second criterion, and then a third one, and so on. This type of problem is denoted as the *r-lexicographic multi-objective problem* (r-LMOP).

To define the problem more precisely we describe the general combinatorial optimization problem with S, the set of feasible solutions, as

$$\min\{f(\boldsymbol{x}) \mid \boldsymbol{x} \in S\} \quad \text{or} \quad \max\{f(\boldsymbol{x}) \mid \boldsymbol{x} \in S\}$$

for the minimization or the maximization version.

We define f for problems given on a weighted graph $G = G(N, A, c)$ with node set N and arc set A having n nodes and m arcs, where appropriate denoted as (i,j), and $c : A \to \Re$ is the weight function. The *sum* objective can be given as

$$\min f(\boldsymbol{x}) = \sum_{a \in \boldsymbol{x}} c(a) \quad \text{or} \quad \max f(\boldsymbol{x}) = \sum_{a \in \boldsymbol{x}} c(a)$$

and the *bottleneck* objective as

$$\min f(\boldsymbol{x}) = \max_{a \in \boldsymbol{x}} c(a) \quad \text{or} \quad \max f(\boldsymbol{x}) = \min_{a \in \boldsymbol{x}} c(a).$$

* Tel.: +31-20-525-4219; fax: +31-20-525-4349.
E-mail address: tonv@fee.uva.nl (A. Volgenant).

0377-2217/02/$ - see front matter © 2002 Elsevier Science B.V. All rights reserved.
PII: S0377-2217(01)00214-4

Now we can define more precisely the r-LMOP as: given r objective functions $f_1(\boldsymbol{x}), f_2(\boldsymbol{x}), \ldots, f_r(\boldsymbol{x})$ and r operators chosen from the set {min sum, max sum, min bottleneck, max bottleneck} first one optimizes $f_1(\boldsymbol{x})$ subject to $\boldsymbol{x} \in S$ according to the first operator, obtaining a set of optimal solutions S_1^*, next one optimizes $f_2(\boldsymbol{x})$ subject to $\boldsymbol{x} \in S_1^*$ according to the second operator, obtaining a set of optimal solutions S_2^*, and so on. Clearly $S_1^* \supseteq S_2^* \supseteq \cdots \supseteq S_r^*$. If $|S_i^*| = 1$ for some value of i with $1 \leqslant i \leqslant r$ the remaining problem(s) can be left out of consideration.

A straightforward approach is to generate S_1^* by applying a k best solutions scheme, see Lawler (1972), that has complexity $O(kn)T(m,n)$ with $T(m,n)$ the complexity of the optimization problem. Then, one enumerates the k best solutions under the second criterion and so on as long as a choice remains. This approach is only effective if a small value of k suffices, even for specific problem types with a better complexity of $O(k)T(m,n)$.

Sokkalingam and Aneja (1998) have analysed a *lexicographic bottleneck criterion*; it is a special case of bottleneck optimization, i.e., in addition to, say, minimizing the largest element of a feasible solution, one is also interested in minimizing the second largest element, the third largest element, and so on. Their iterative solution approach can be seen as an LMOP with alternating min bottleneck and min sum criteria; in the odd steps of it a sum problem is solved for an auxiliary weight function with only 0- and 1-coefficients. By exploiting the structure of this specialized scheme they sometimes obtain an improved complexity.

We will consider a general iterative scheme for the r-LMOP in Section 3. We apply the scheme to linear assignment in Section 4, to path problems in Section 5 and to spanning tree problems in Section 6, obtaining a better complexity than the known approach for r-LMOP. In Section 2 we first describe this approach, which is based on coefficient scaling.

2. The scaling approach

Sherali (1982) described an algorithm for r-LMOP with the operator max sum that exploits sum optimization on scaled cost coefficients; we outline this scaling approach to derive the complexity of it.

The quantity $\mathrm{UB}(C_r)$ is defined as an upper bound on criterion f_r given by the cost coefficients in the corresponding cost matrix C_r; for a combinatorial problem and cost weights c_i $(i = 1, \ldots, m)$ it is in general given as $\mathrm{UB}(C) = \sum_{i=1}^{m} |c_i|$.

The three steps of the scaling approach are (given a value $r > 1$ indicating the number of objectives):

Initialization	let $\rho := r$, $F_r := C_r$;
Iteration	$\rho := \rho - 1$; compute $\lambda_\rho = 1 + \mathrm{UB}(F_{\rho+1})$ and $F_\rho = F_{\rho+1} + \lambda_\rho C_\rho$;
Termination	if $\rho > 1$ then go to *Iteration*; solve the one-step problem with cost coefficient matrix F_1 and stop.

Since the weights of F_1 grow exponentially in the worst case, an elementary operation (e.g., addition) has to be considered proportional to the logarithm of the largest incurred weight. The largest weight $s(e)$ satisfies $\log s(e) = O(\log m^r) = O(r \log m)$. Assuming that an elementary operation (e.g., addition) on the given weights takes constant time, such an operation on the weights of F_1 takes $O(r \log m)$ time. As a consequence the scaling approach has complexity $O(r \log m)\ T(m,n)$ and $O(r \log n)T(m,n)$ for the considered problem types with $O(m) = O(n)$ arcs or edges.

For the r-LMOP with sum criteria and one bottleneck criterion the growth of coefficients is even stronger for the scaling approach. To solve a bottleneck problem as a sum problem on scaled coefficients the scaled values have to be up to 2^{n^2} or n^m; this leads to an extra factor of $O(n^2)$ or $O(m \log n)$ in the complexity. For a mixture of criteria with r_b bottleneck criteria this factor becomes r_b times larger.

Sherali (1982) solved the linear assignment 3-LMOP on the example

$$C_1 = \begin{pmatrix} 2 & 4 & 2 \\ 1 & 2 & 1 \\ 3 & 1 & 0 \end{pmatrix}, \quad C_2 = \begin{pmatrix} 3 & 1 & 4 \\ 2 & 5 & 3 \\ 1 & 2 & 1 \end{pmatrix},$$

$$C_3 = \begin{pmatrix} 4 & 2 & 2 \\ 3 & 2 & 3 \\ 2 & 5 & 2 \end{pmatrix}, \quad F_1 = \begin{pmatrix} 129 & 217 & \mathbf{134} \\ \mathbf{69} & 141 & 76 \\ 165 & \mathbf{71} & 9 \end{pmatrix}.$$

He obtained $F_1 = 52C_1 + F_2 = 52C_1 + 7C_2 + C_3$ by tightening the bound UB to the sum of the differences between the maximum and the minimum entries in the rows of the cost matrix C. The optimal assignment – in **bold** – is $(3,1,2)$ with $z^*(F_1) = 274$ and $z^* = (4,8,10)$.

In this approach it is more efficient to solve first a linear assignment problem on the matrices C as well as F and to replace them by the matrices of reduced costs denoted as C^{red} and F^{red}; such a reduction does not change the difference between the criterion values of the two linear assignment solutions. This improvement has no influence on the complexity of the scheme; it results generally in a smaller largest coefficient in the reduced matrix F_1^{red}: in the example one finds

$$F_1^{\text{red}} = 42C_1^{\text{red}} + 7C_2^{\text{red}} + C_3^{\text{red}}$$

and as maximal coefficient 126 (instead of 217).

In general, the increase of the computational complexity by scaling is caused by the exponential growing of the parameters in the scaled problem with respect to the original parameters, increasing the complexity of an elementary addition operation. We shortly discuss the practical implications of this phenomenon. Computers perform the addition operation in constant time for all the values within the range of a given type (integer, double); for personal computers, e.g., suppose the integer data type uses 2 bytes and the double data type 4 bytes, so that integer values can be treated up to about $\pm 32{,}000$ and up to about $\pm 2 \times 10^9$. Thus, in practice the scaling approach needs an extra implementation effort when the scaled cost coefficients exceed these ranges.

To give an illustration, consider a linear assignment problem with given cost coefficients on the interval $\langle 0, \mu \rangle$, $n = 1000$ (an easily solvable size for this type of problem) and $r = 4$; then the scaled coefficients can be up to $(1000\mu)^3 = 10^9\mu$, which gives an overflow for $\mu = 100$. For the r-LMOP with sum criteria and, say, one bottleneck criterion, the scaled values have to be up to 2^{n^2} or n^m as mentioned above. For an example with, say, $n = 1000$ and $m = 10{,}000$, this leads to scaled coefficients that are intractable without a substantial extra implementation effort and extra computation time and memory requirements.

3. A general scheme

We describe an iterative approach for combinatorial optimization problems solving a series of sum and bottleneck optimization problems on reduced sets of arcs. The scheme is suited for a mixture of maximizing and minimizing a series of sum and bottleneck criteria.

For a considered problem with a given set of arcs A we define $S(A)$ as the set of the feasible solutions on A and $S^*(A)$ as the set of the optimal solutions on A, while the optimal criterion value is denoted by z^*. We introduce quantities $\delta(a)$ for the arcs in set A, defined as

$$\delta(a) = \begin{cases} 1 & \text{if the increment of the optimal criterion value is positive under the extra constraint that arc } a \text{ has to be in an optimal solution,} \\ 0 & \text{otherwise.} \end{cases} \tag{1}$$

The set A^* is defined as $A^* = \{(i,j) \mid \delta(i,j) = 0\}$. We prove the following theorem for these quantities.

Theorem. *For a given problem with arc set A the set of optimal solutions $S^*(A)$ is equal to $S(A^*)$, the set of feasible solutions on A^*.*

Proof. Suppose an optimal solution σ^* on A, i.e., $\sigma^* \in S^*(A)$, contains an arc $i^* \to j^*$ with $(i^*, j^*) \notin A^*$, then $\delta(i^*, j^*) = 1$, contradicting the optimality of the arc (i^*, j^*) and also of σ^*, so σ^* contains only arcs that are in A^*; we can conclude that $\sigma^* \in S(A^*)$.

The other way around: let a feasible solution $\sigma \in S(A^*)$ contain $i \to j$, i.e., $(i,j) \in A^*$ and $\delta(i,j) = 0$. As this is true for all arcs in σ, we have that $\sigma \in S(A^*)$.

Given this theorem one can solve the r-LMOP by applying a simple solution scheme:

Initialization	let $\rho := r+1$; $A_r := A$;
Iteration	$\rho := \rho - 1$;
	determine A^*_ρ and $S^*(A_\rho)$;
Termination	if $\rho > 1$ and $\lvert S^*(A_\rho)\rvert > 1$ then go to *Iteration*
	otherwise the r-LMOP has been solved.

If the complexity of determining A^* together with the quantities $\delta(a)$ is at most equal to the complexity of the optimization problem, then the complexity of the r-LMOP is $O(r)T(m,n)$, an improvement over the scaling approach of a factor $\log n$.

For the bottleneck criterion and a given combinatorial optimization problem the quantities $\delta(a)$ can be chosen as $\delta(a) = 1$ if $c(a) > z^*$ and 0 otherwise. Given the value of z^* the determination of all the δ-values has complexity $O(m)$.

We will show that this solution scheme can be applied efficiently to the (sum) linear assignment, to the (sum) minimum spanning tree and to the (sum) shortest path LMOP. In case of undirected problem types, such as the minimal spanning tree problem, we have edges instead of arcs and we denote the set of edges as E; the matrix of the cost coefficients then is symmetric.

For the last two problem types we will also consider an alternative bottleneck criterion, denoted as *shortest bottleneck*; it is applied when one is interested in optimal bottleneck solutions that are also the shortest among all the optimal bottleneck solutions. It can be seen as a bottleneck criterion followed by a min sum criterion on the same cost coefficients.

In the implementation of the general scheme one can terminate it prematurely if $\lvert S(A^*)\rvert$ is small; then it is more efficient to compute the values for the last criteria straightforwardly by enumeration. By the forward structure of the scheme it can profit from the case $\lvert S(A^*)\rvert = 1$, i.e., the remaining problem instance has a unique solution, and no more computations are necessary, in contrast to the backward structure of the scaling approach, which gives no smaller scaled coefficients and saves no computational effort.

4. Linear assignment

Say one seeks to assign jobs to machines to minimize cost, delay, machine wear, error in accuracy, etc. in some priority order, then one has to solve a linear assignment r-LMOP. Caron et al. (1999) consider personnel assignment under seniority constraints for given seniority classes. An assignment of persons to jobs satisfies such constraints if no jobless person can be given a job without a person of the same or higher seniority class becoming jobless. This type of problem can be seen as a linear assignment r-LMOP by defining auxiliary criterion functions (with 0–1 criterion weights) for the successive seniority classes; the same holds for assignments under the job priority constraints that they also consider. Their solution method is based on scaling.

To apply the general scheme of the previous section we can exploit *hard dual values* $h(i,j)$, see Kindervater et al. (1985); they introduced these values $h(i,j)$ as the increment of the optimal criterion value under the extra constraint that arc (i,j) has to be in an optimal solution. They show that by solving a shortest path problem for each node i $(i = 1,\ldots,n)$ a dual solution $(\boldsymbol{u}^i, \boldsymbol{v}^i)$ can be determined with the property that $h(i,j) = c(i,j) - u^i_i - v^i_j$ $(j = 1,\ldots,n)$. Now we have for δ the choice $\delta(i,j) = 1$ if $h(i,j) > 0$ and $\delta(i,j) = 0$ otherwise.

Thus the complexity is $O(rn^3)$ for the sum case, as the computation of the hard dual values consists of solving n times a shortest path problem and this costs $O(n^3)$, the same complexity as for the sum linear assignment problem.

The bottleneck linear assignment problem has complexity $O(n^{2.5})$, see Punnen and Nair (1994); so the bottleneck linear assignment r-LMOP has complexity $O(rn^{2.5})$.

In the implementation of the bottleneck and the sum criterion one can decrease the problem size if only one arc remains in a row or a column.

We illustrate the approach on the example of Sherali (1982, Section 2); the hard dual values are successively given by H_1 and H_2:

$$H_1 = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 0 \\ 3 & 0 & 0 \end{pmatrix}, \quad H_2 = \frac{1}{3}\begin{pmatrix} 0 & - & 0 \\ 0 & 2 & 0 \\ - & 0 & 1 \end{pmatrix},$$

$$C_3^* = \begin{pmatrix} 4 & - & \mathbf{2} \\ \mathbf{3} & - & 3 \\ - & \mathbf{5} & - \end{pmatrix}.$$

Now it is sufficient to solve the LAP on C_3^* resulting in the optimal assignment $(3,1,2)$ – in **bold** – with $z^* = (0,0,10)$; in original weights

$z^* = (2+1+1=4, 4+2+2=8, 10)$.

As the optimal assignments on the matrices H are always on the zero coefficients, it is easy to give an upper bound, say k, to the number of optimal solutions, e.g., the product of the number of zeros in the rows (or the columns). This bound can be used in the straightforward approach of Section 1. For the example of H_1 one finds as bound $2 \times 3 \times 2 = 12$, and this approach is not very efficient. It is better to determine k as a sum of the bounds for the number of solutions with either the assignment $(1,1)$ or the assignment $(1,3)$ in it; one finds $2+1=3$ as a better value for k.

5. Shortest path

We restrict the analysis to non-negative distances; for the problem type where negative distances are allowed the analysis is similar, although the complexity is larger. For the *sum* criterion we consider the set of all paths from a specific node $s \in N$; for the *bottleneck* criterion we also treat the case of one specified end node t.

Letting $d(i)$ denote the shortest distance from node s to node i, the optimality conditions for a shortest path are $d(j) \leqslant d(i) + c(i,j)$ for every arc $(i,j) \in A$. With the quantity

$$\delta(i,j) = \begin{cases} 1 & \text{if } d(j) - d(i) < c(i,j), \\ 0 & \text{otherwise,} \end{cases} \tag{2}$$

the set A^* can be determined. Note that A^* contains all the arcs of the shortest paths starting in s and ending in each of the other nodes.

One can find the bottleneck paths starting in s and ending in each of the other nodes by exploiting the optimality conditions for each path

$$b(j) \leqslant \max\{b(i), c_{ij}\} \quad \text{for every arc } (i,j) \in A,$$

where the label $b(i)$ of node i denotes the bottleneck value of the path from node s to node i. Then the set A^* can be determined by defining for each arc (i,j)

$$\delta(i,j) = \begin{cases} 1 & \text{if } \max\{c(i,j), b(i)\} > b(j), \\ 0 & \text{otherwise.} \end{cases} \tag{3}$$

We can solve both the last-mentioned bottleneck and the shortest path problem by variations of the Dijkstra (1959) algorithm, which produces also the corresponding labels b respectively d. The complexity is $O(n^2)$ for the min sum, the min bottleneck and the max bottleneck criterion, leading to complexity $O(rn^2)$ for the r-LMOP variant of these criteria. It is easy to check that by using the b-labels the obtained bottleneck solutions are also the shortest.

In the undirected case the bottleneck path from node s to one given node t has complexity $O(m)$, see, e.g., Punnen (1991), and the complexity of the r-LMOP is $O(rm)$ by applying the general scheme. The same complexity holds for the shortest path r-LMOP with only 0–1 cost weights because then one can use Dijkstra's (1959) algorithm in the implementation of Dial (1969). As a consequence the lexicographic bottleneck problem can be solved as an r-LMOP with $r = 2\min\{k,n\}$ obtaining the same complexity as Sokkalingam and Aneja (1998), i.e., $O(\min\{km, nm\})$.

The max sum criterion is not polynomial, as the longest path problem is equivalent to the travelling salesman problem and as a consequence the r-LMOP for paths is not polynomially solvable if one of the criteria is the max sum criterion.

With respect to the straightforward approach mentioned in Section 1: Lawler (1972) gives complexity $O(kn^3)$ for the k best solutions over all shortest paths also on matrices with negative distances. Azevedo et al. (1993) have given an alternative approach for sparse problem instances with

a complexity $O(km)$; they claim that their algorithm performs often better in practice than the Dreyfus (1969) algorithm that enables a complexity of $O(kn \log n)$ exclusive of an initial computational effort with higher complexity. Eppstein (1998) has developed an algorithm with complexity $O(m + n \log n + kn)$. The r-LMOP has complexity $O(rn^2)$; in practice this approach is the best one, unless a small value of k is sufficient.

6. Minimum spanning tree

For ease of presentation we first introduce some notations and known notions.

Without loss of generality we assume E to be a complete set of edges; as far as necessary we add edges with a weight M given as a sufficiently large value.

- An edge of E in a tree T is called a *branch* of T and an edge of E not in T is called a *chord* of T.
- The *fundamental path* of a chord (r, s) is the unique chain of T between r and s.
- The *fundamental cutset* of a branch (p, q) is the cutset of the two connected subgraphs of T that remain when (p, q) is removed.
- For two disjoint subsets N_1 and N_2 the cutset of edges in E is defined by $\{(i, j) \mid i \in N_1,\ j \in N_2\}$.

Necessary and sufficient conditions for a spanning tree T to be minimal are, see, e.g., Ford and Fulkerson (1962):

1. *Every chord of the tree is at least as long as any branch in its fundamental path*;
2. *Every branch of the tree is at least as short as any chord in its fundamental cutset*.

Because of (2) a minimum spanning tree is also a minimal bottleneck tree, as well as a minimal lexicographic bottleneck tree.

Given T^*, a minimal spanning tree on G, the quantity $\delta(i, j)$ can be chosen as

$$\delta(i,j) = \begin{cases} 1 & \text{if } c(i,j) - c(r_{ij}, s_{ij}) > 0 \\ & \quad \text{for } (i,j) \text{ a chord of } T^*, \\ 0 & \text{otherwise,} \end{cases} \tag{4}$$

with (r_{ij}, s_{ij}) a longest branch on the fundamental path of (i, j).

Implementation of the determination of all the δ-values can be done in $O(n^2)$, see e.g., Volgenant and Jonker (1983), by traversing the chords of the tree in a directed order, following from the topology of the tree. Therefore, the total complexity of the minimal spanning tree r-LMOP is $O(rn^2)$.

The bottleneck spanning tree can be determined in $O(m)$, see Camerini (1978), just as all the δ-values; thus the complexity of the bottleneck spanning tree r-LMOP is $O(rm)$.

The Steiner version of the bottleneck spanning tree spans a given subset of the node set minimizing the bottleneck criterion; the problem is solvable in $O(m)$, see Duin and Volgenant (1997). Hence the Steiner bottleneck tree r-LMOP has complexity $O(rm)$.

7. Some final remarks and conclusions

We make some final remarks.

- The described LMOP approach can be applied to the mixed LMOP given by first a number of minimal or bottleneck *spanning tree* problems and then a number of shortest or bottleneck *path* problems; as the number of arcs of a shortest or bottleneck path is not known beforehand, the path problems have to be at the end of the series of subproblems.
- A similar remark holds for the mixed LMOP given by spanning tree problems, followed by Steiner bottleneck tree problems.
- If the resulting r-LMOP solution is not yet unique, then one can consider further desired properties of an optimal solution, e.g., in case of the shortest path r-LMOP a solution with the smallest number of arcs; one finds such a solution by adding an extra sum criterion with weights 1 on the arcs of the arc set; in the last step one can apply even criteria for which δ-values are not known or scaling is impossible, such as balanced optimization or minimization of deviations, see, e.g., Duin and Volgenant (1991).
- It can be interesting to search for other problems that can be solved by the r-LMOP solution scheme.

We conclude that one can solve the r-LMOP for linear assignment, shortest paths and minimal spanning trees in $O(r)T(m, n)$ for the criteria

min sum and min bottleneck, an improvement over the scaling approach of a factor that is $\log n$ in case only sum criteria are involved, while the factor is at least $n \log n$ larger in case also bottleneck criteria are involved.

Acknowledgements

I am grateful to an anonymous referee who gave helpful comments that have led to improvement of this paper.

References

Azevedo, J.A., Santos Costa, M.E.O., Silvestre Madeira, J.J.E.R., Vieira Martins, E.Q., 1993. An algorithm for the ranking of shortest paths. European Journal of Operational Research 69, 97–106.

Camerini, P., 1978. The min–max spanning tree problem and some extensions. Information Processing Letters 7, 10–14.

Caron, G., Hansen, P., Jaumard, B., 1999. The assignment problem with seniority and job priority constraints. Operations Research 47, 449–453.

Dial, R., 1969. Algorithm 360: Shortest path forest with topological ordering. Communications of ACM 12, 632–633.

Dijkstra, E.W., 1959. A note on two problems in connexion with graphs. Numerische Mathematik 1, 269–271.

Dreyfus, S.E., 1969. An appraisal of some shortest-path algorithms. Operations Research 17, 395–412.

Duin, C.W., Volgenant, A., 1991. Minimum deviation and balanced optimization: A unified approach. Operations Research Letters 10, 43–48.

Duin, C.W., Volgenant, A., 1997. The partial sum criterion for Steiner trees in graphs and shortest paths. European Journal of Operational Research 97, 172–182.

Eppstein, D., 1998. Finding the k shortest paths. SIAM Journal of Computing 28, 652–673.

Ford, L.R., Fulkerson, D.R., 1962. Flows in Networks. Princeton, NJ.

Kindervater, G., Volgenant, A., de Leve, G., van Gijlswijk, V., 1985. On dual solutions for the linear assignment problem. European Journal of Operational Research 19, 76–81.

Lawler, E.L., 1972. A procedure for computing the k best solutions to discrete optimization problems and its applications to the shortest path problem. Management Science 18, 401–407.

Punnen, A.P., 1991. A linear time algorithm for the maximum capacity path problem. European Journal of Operational Research 53, 402–404.

Punnen, A.P., Nair, P.K.P., 1994. Improved complexity bound for the maximum cardinality bottleneck bipartite matching problem. Discrete Applied Mathematics 55, 91–93.

Sherali, H.D., 1982. Equivalent weights for lexicographic multi-objective programs: Characterizations and computations. European Journal of Operational Research 11, 367–379.

Sokkalingam, P.T., Aneja, Y.P., 1998. Lexicographic bottleneck combinatorial problems. Operations Research Letters 23, 27–33.

Volgenant, A., Jonker, R., 1983. The symmetric traveling salesman problem and edge exchanges in minimal 1-trees. European Journal of Operational Research 12, 394–403.